



# Diseño de Compiladores I

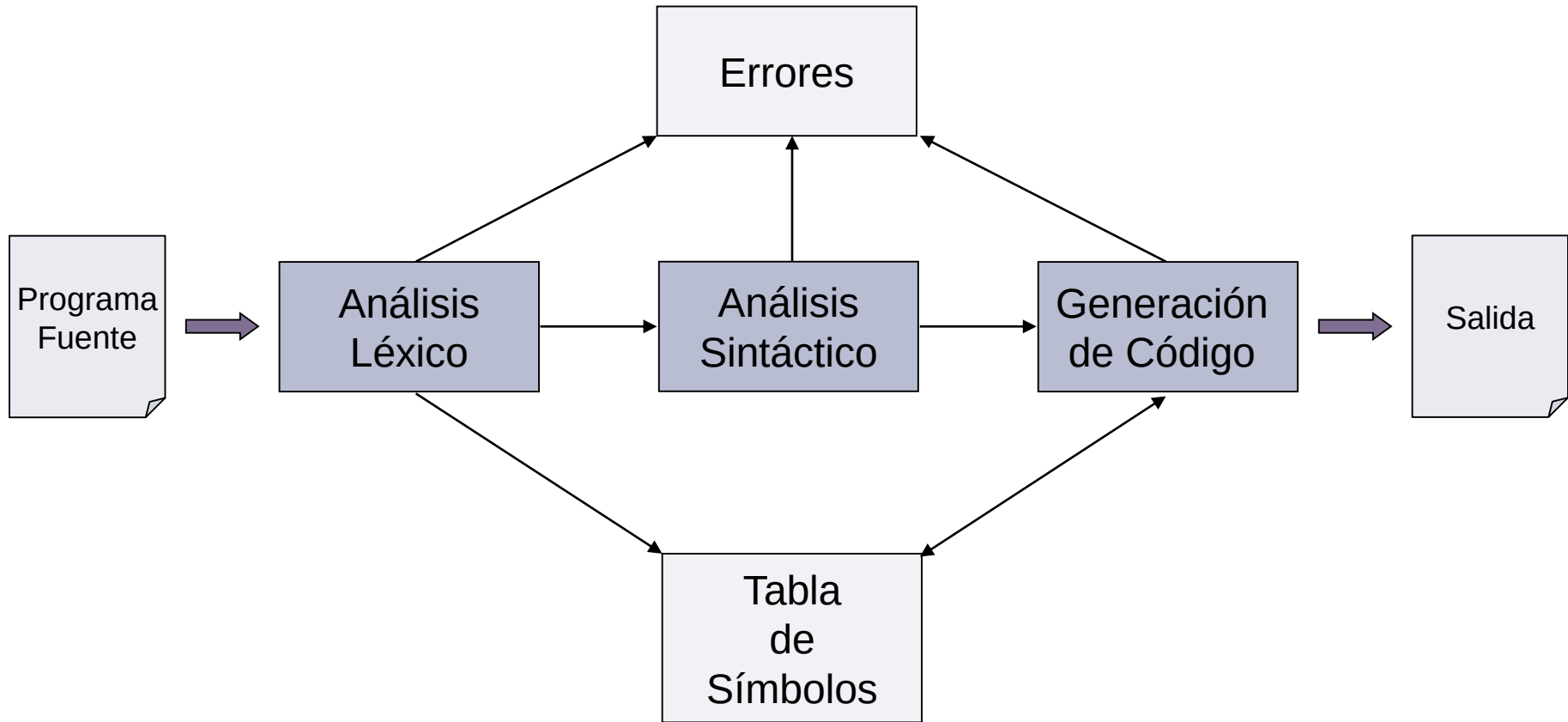
## Compiladores e Intérpretes



Análisis Léxico

# Fases de la Compilación

---



# Análisis Léxico

---

- ▶ Lee el programa fuente.
- ▶ Agrupa los caracteres en unidades llamadas **tokens**.

**token**: Secuencia de caracteres que forman una unidad significativa



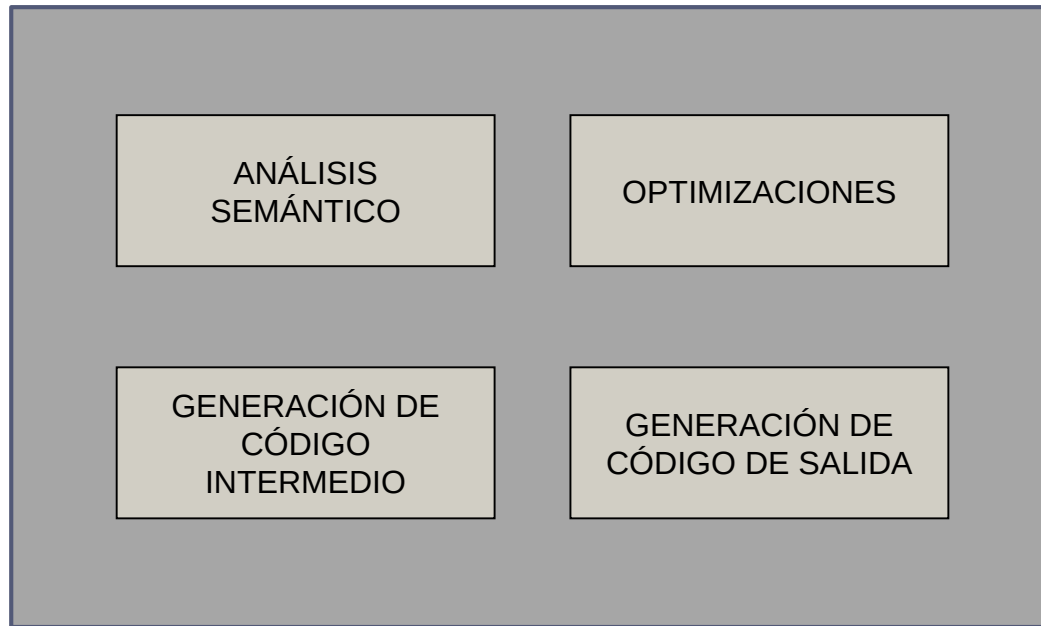
# Análisis Sintáctico

---

Agrupar los tokens del programa fuente en frases gramaticales que el compilador usará en las siguientes etapas.

# Fases de la Compilación

---



Generación de Código

# Generación de Código

---

## Análisis Semántico

- ▶ Chequea reglas que no pueden ser capturadas por la gramática, pero que pueden ser verificadas en tiempo de compilación. Estas reglas corresponden a la semántica estática del lenguaje.

## Generación de Código Intermedio

- ▶ Se construye una representación del código fuente como un programa escrito para ser ejecutado en una máquina abstracta.

## Optimización

- ▶ Transforma la representación actual del código en una nueva versión que logra el mismo resultado más eficientemente.

## Generación de Código propiamente dicho

- ▶ Se traduce la representación intermedia del programa fuente al código nativo de la máquina objetivo.
- ▶ El código generado efectuará el chequeo de las reglas de semántica dinámica del lenguaje, que no pudieron ser verificadas durante la compilación.

---

## ► Tabla de Símbolos

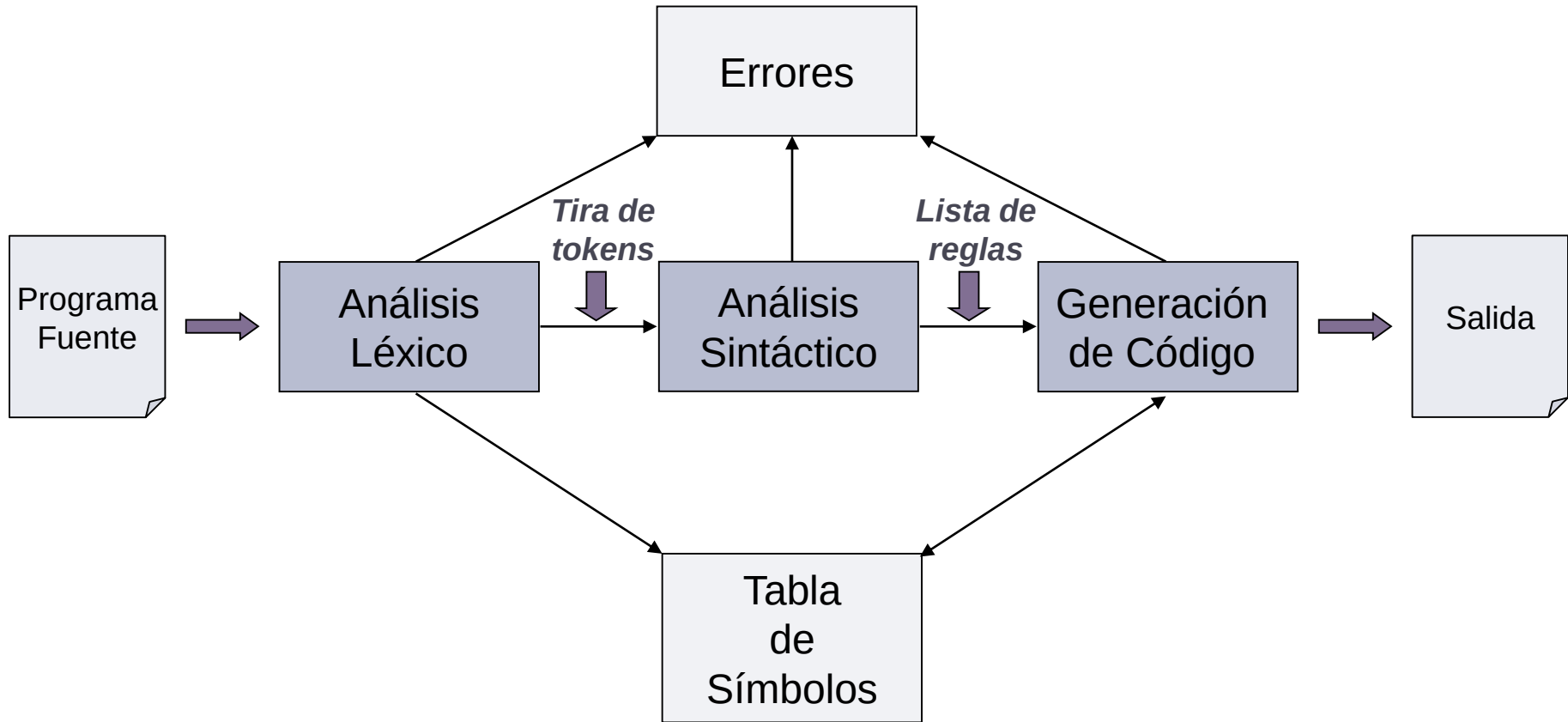
- Es una estructura de datos que contiene un registro para cada símbolo utilizado en el código fuente, con campos que contienen información relevante para cada símbolo (atributos). Sólo se registran en la Tabla aquellos símbolos para los cuales pueden existir distintos lexemas para un mismo token.

## ► Errores

- Cada una de las etapas del Compilador puede detectar errores que son informados al programador.
- Un buen compilador no debería terminar su ejecución al detectar un error, sino que debería recuperarse y continuar con la compilación.

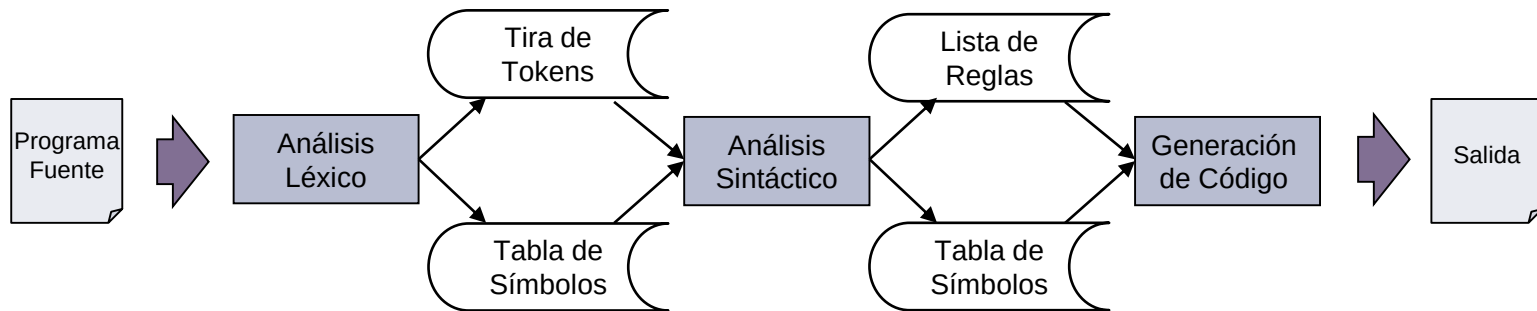
# Fases de la Compilación

---



# Interacción Analizador Léxico – Analizador Sintáctico – Generador de Código

## ► Batch



## ► Concurrente

### ► Ejemplo: gcc

c1 | c2 | c3  
(AL) (AS) (GC)

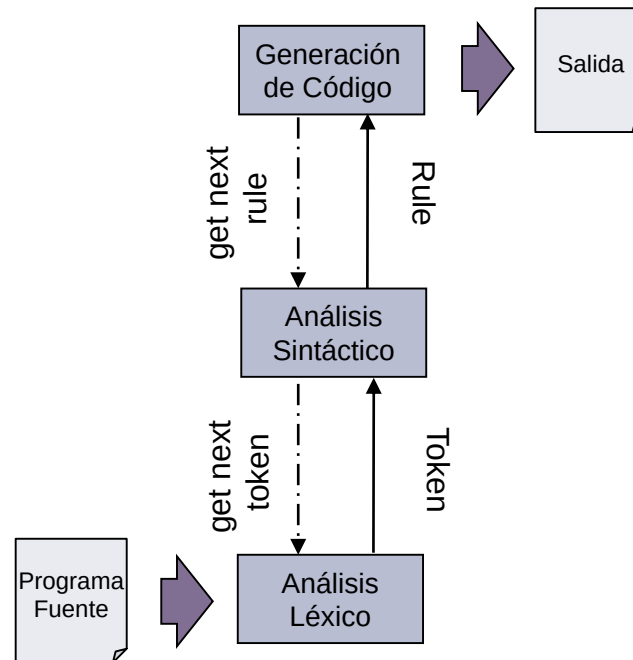
# Interacción Analizador Léxico – Analizador Sintáctico – Generador de Código

---

## ► Monolítico

- El Generador de Código es el main

El A.S. entrega una regla cuando el G.C. se la pide, y el A.L. entrega un token cuando el A.S. se lo pide



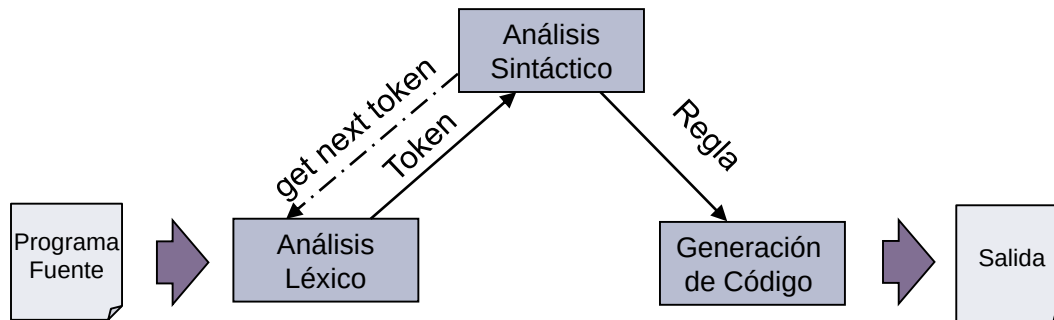
# Interacción Analizador Léxico – Analizador Sintáctico – Generador de Código

---

## ► Monolítico

- El Parser (A.S.) es el main:

El Analizador Sintáctico va pidiendo tokens al Léxico, y cuando tiene una regla le pide al G.C. que genere código para ella.



*Esta es la opción que utiliza Yacc*

---

# ¿Por qué un Analizador Léxico?

# Tokens

---

- ▶ Identificadores
- ▶ Constantes
- ▶ Cadenas de caracteres
- ▶ Palabras reservadas
- ▶ Operadores
- ▶ Comparadores
- ▶ etc.

# Tokens

---

- ▶ Los tokens se diferencian de la cadena de caracteres que representan.
- ▶ La cadena de caracteres es el **Lexema** o valor léxico.
  - ▶ Existen tokens que se corresponden con un único lexema
    - ▶ Ejemplo: Palabra Reservada IF
  - ▶ Existen tokens que pueden representar lexemas diferentes
    - ▶ Ejemplos:
      - Identificador Plazo, Identificador Tasa
      - Constante 100, Constante 1.5E-02
      - Cadena de caracteres: “Hola mundo”, “Esto es una cadena”

# Análisis Léxico

## Ejemplo

---

```
if Plazo >= 30  
  then Tasa := Base + Recargo / 100  
  else Tasa := Base
```

# Análisis Léxico

## Ejemplo

---

[if] [Plazo] [>=] [30]

[then] [Tasa] [:=] [Base] [+] [Recargo] [/] [100]

[else] [Tasa] [:=] [Base]

# Tokens

---

Para las implementaciones en que el Analizador Sintáctico invoca al Léxico cada vez que necesita un token:

- ▶ El Analizador Léxico hace una correspondencia entre cada tipo de token y un número entero.
- ▶ El Analizador Léxico entrega al Analizador Sintáctico los números enteros que corresponden a cada tipo de token detectado en el código fuente.

Para las implementaciones en que el Analizador Léxico entrega al Sintáctico la lista de tokens:

- ▶ La lista incluirá directamente los tokens.

# Tokens

---

| Tipo de token | Identificación del tipo de token |
|---------------|----------------------------------|
| ID            | 27                               |
| CTE           | 28                               |
| IF            | 59                               |
| THEN          | 60                               |
| ELSE          | 61                               |
| +             | 70                               |
| /             | 73                               |
| >=            | 80                               |
| :=            | 85                               |

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

[if] [Plazo] [ $\geq$ ] [30]

[then] [Tasa] [:=] [Base] [+] [Recargo] [/] [100]

[else] [Tasa] [:=] [Base]

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

[59] [Plazo] [ $\geq$ ] [30]

[then] [Tasa] [:=] [Base] [+] [Recargo] [/] [100]

[else] [Tasa] [:=] [Base]

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

[59] [27] [ $\geq$ ] [30]

[then] [Tasa] [:=] [Base] [+] [Recargo] [/] [100]

[else] [Tasa] [:=] [Base]

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

**[59] [27] [80] [30]**

**[then] [Tasa] [:=] [Base] [+] [Recargo] [/] [100]**

**[else] [Tasa] [:=] [Base]**

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

**[59] [27] [80] [28]**

[then] [Tasa] [:=] [Base] [+] [Recargo] [/] [100]

[else] [Tasa] [:=] [Base]

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

**[59] [27] [80] [28]**

**[60] [27] [85] [27] [70] [27] [73] [28]**

**[61] [27] [85] [27]**

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

**[IF] [ID] [GT] [CTE]**

**[THEN] [ID] [ASIG] [ID] ['+'] [ID] ['/'] [CTE]**

**[ELSE] [ID] [ASIG] [ID]**

# Tokens

---

- ▶ Puesto que un token puede representar más de un lexema, el A.L. debe enviar información adicional al A.S., en forma de atributo/s. Esa información será usada por el G.C.
- ▶ Por cada token detectado, el A.L. entrega un par:  
    <token, atributo>
- ▶ En principio, ese atributo sería el lexema.

# Análisis Léxico

## Ejemplo

---

if Plazo >= 30

then Tasa := Base + Recargo / 100

else Tasa := Base

[59, 'IF'] [27, 'Plazo'] [80, '>='] [28, '30']

[60, 'THEN'] [27, 'Tasa'] [85, ':=' ] [27, 'Base'] [70, '+' ] [27, 'Recargo'] [73, '/' ] [28, '100']

[61, 'ELSE'] [27, 'Tasa'] [85, ':=' ] [27, 'Base']

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

[59, 'if'] [27, 'Plazo'] [80, 'if'] [28, '30']  
[60, 'then'] [27, 'Tasa'] [85, 'then'] [27, 'Base'] [70, 'then'] [27, 'Recargo']  
[73, 'then'] [28, '100']  
[61, 'else'] [27, 'Tasa'] [85, 'else'] [27, 'Base']

# Tokens

---

## ENTONCES:

- ▶ **Sólo para aquellos tokens que pueden representar más de un lexema**, el A.L. necesita entregar un par:

`<token, atributo>`

En principio, ese atributo será el lexema.

- ▶ **Para aquellos tokens que representan un solo lexema**, el A.L. puede entregar sólo el token.

# Análisis Léxico

## Ejemplo

---

if Plazo  $\geq$  30

then Tasa := Base + Recargo / 100

else Tasa := Base

[59] [27, 'Plazo'] [80] [28, '30']

[60] [27, 'Tasa'] [85] [27, 'Base'] [70] [27, 'Recargo'] [73] [28, '100']

[61] [27, 'Tasa'] [85] [27, 'Base']

# Análisis Léxico

## Ejemplo

---

```
if Plazo >= 30
then Tasa := Base + Recargo / 100
else Tasa := Base
```

```
[59] [27,'Plazo'] [80] [28,'30']
[60] [27,'Tasa'] [85] [27,'Base'] [70] [27,'Recargo'] [73] [28,'100']
[61] [27,'Tasa'] [85] [27,'Base']
```

- **Información necesaria para el Análisis Sintáctico:**

**IF ID >= CTE THEN ID := ID + ID / CTE ELSE ID := ID**

- **Los atributos se necesitan durante la Generación de Código**

# Atributos de los tokens

---

- ▶ Lexema, nro. de línea en que aparece el token, etc.
- ▶ En la práctica, la información adicional para cada token se almacena en una **Tabla de Símbolos**, y el atributo entregado es el puntero o referencia a la entrada correspondiente en la Tabla de Símbolos.

# Tabla de Símbolos

---

Es una estructura de datos que contiene un registro para cada identificador (y todo otro token que pueda representar más de un lexema) utilizado en el código fuente, con campos que contienen información relevante para ese símbolo (atributos).

# Tabla de Símbolos

---

- ▶ Cuando el Análisis Léxico detecta un token de tipo identificador, u otro token que pueda representar más de un lexema, lo ingresa en la Tabla de Símbolos.  
*(El alta en la Tabla de Símbolos podría hacerse durante la Generación de Código)*
- ▶ Durante la Generación de Código se ingresa información para los atributos de los símbolos, y se usa esa información de diversas maneras.
- ▶ Durante la Generación de Código puede ser necesario incorporar nuevas entradas a la Tabla de Símbolos.

# Análisis Léxico: Funciones

---

## Principales:

- ▶ Reconocer tokens
- ▶ Informar errores léxicos

## Secundarias:

- ▶ Eliminar comentarios
- ▶ Eliminar blancos, tabulaciones, saltos de línea
- ▶ Llevar la cuenta de los saltos de línea, para correlacionar los mensajes de error con el programa fuente

# Construcción del Analizador Léxico

---

- ▶ Construir un diagrama que represente la estructura de los tokens del programa fuente.
- ▶ Convertir el diagrama en un programa.

# Autómata Finito

---

- ▶  $AF = \{ Q, S_e, \delta, q_0, F \}$ 
  - ▶  $Q$  : Conjunto finito de estados
  - ▶  $S_e$  : Conjunto finito de símbolos de entrada
  - ▶  $\delta$  : Función de transición
  - ▶  $q_0$  : Estado inicial
  - ▶  $F$  : Conjunto de estados finales

# Gramática independiente de Contexto

---

- ▶ Conjunto de símbolos terminales: los tokens.
- ▶ Conjunto de no terminales.
- ▶ Conjunto de producciones donde cada producción consiste de un no terminal, o lado izquierdo, una flecha y una secuencia de tokens y/o no terminales, o lado derecho.
- ▶ Designación de uno de los no terminales como símbolo de inicio.

# Gramáticas regulares

---

- ▶ Todo no terminal se define en base a terminales, o en base a un NT y un terminal

$A \rightarrow a \mid Aa$  (recursiva a izquierda)

$A \rightarrow a \mid aB$  (recursiva a derecha)

# Diagrama de Transición de estados

---

- Una gramática regular se puede representar mediante un diagrama de transición de estados

Dado:

$$T = \{a,b,c\} \quad NT = \{A,B,C,S\}$$

Gramática:

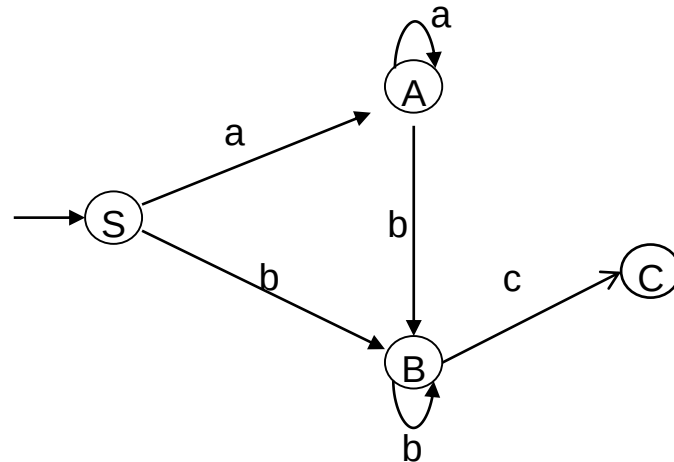
$$A \rightarrow Sa \mid Aa$$

$$B \rightarrow Sb \mid Bb \mid Ab$$

$$C \rightarrow Bc$$

$$S \rightarrow \varepsilon$$

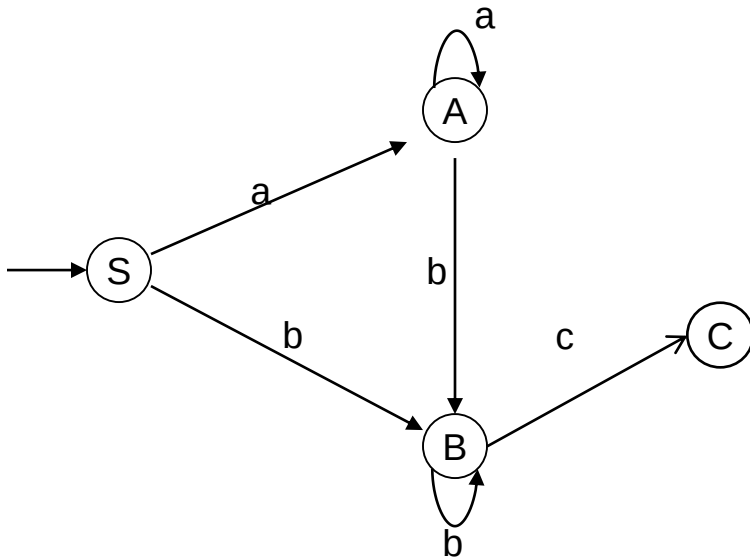
Diagrama de Transición de Estados



# Matriz de Transición de Estados

- El diagrama de transición de estados se puede representar mediante una matriz de transición de estados

Diagrama de Transición de estados



Matriz de Transición de Estados

|   | a | b | c |
|---|---|---|---|
| S | A | B |   |
| A | A | B |   |
| B |   | B | C |
| C |   |   |   |

# Analizador Léxico: Construcción

---

- ▶ Identificadores: cadena que comienza con una letra y continúa con letras y/o dígitos (las letras pueden ser mayúsculas o minúsculas)
- ▶ Constantes enteras (secuencia de dígitos)
- ▶ Comentarios tipo C `/* ... */`
- ▶ Operadores aritméticos `+` `-` `*` `/`
- ▶ Comparadores `>` `>=` `<` `<=` `==` `<>`
- ▶ Palabras reservadas IF, ELSE, WHILE

# Analizador Léxico: Construcción

---

- ▶ Se construye el diagrama de transición de estados que represente la estructura de cada uno de los tokens del lenguaje.

# Análisis Léxico: Gramática

---

- ▶ **Identificador:** cadena que comienza con una letra y continúa con letras y/o dígitos (las letras pueden ser mayúsculas o minúsculas)

ID: letra ( letra | digito )\*

- ▶ **Constante entera:** secuencia de dígitos

CTE: digito +

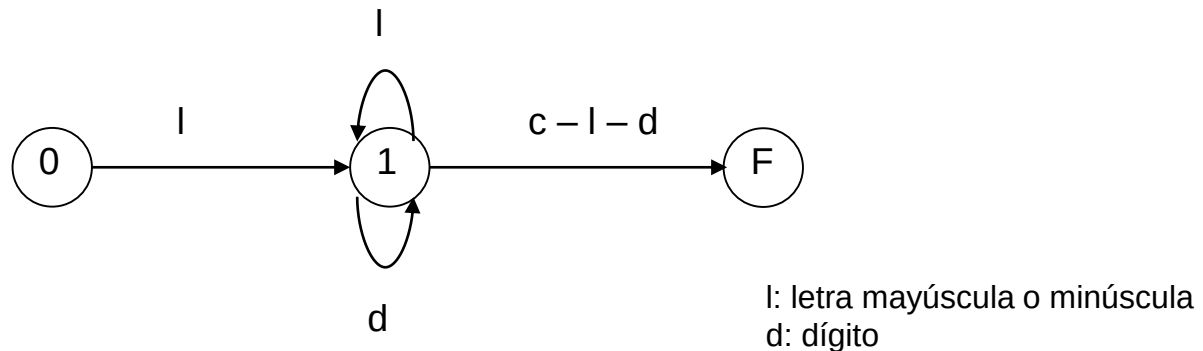
- ▶ Reglas Léxicas:

- ▶  $\langle \text{ID} \rangle \rightarrow \langle \text{letra} \rangle \mid \langle \text{ID} \rangle \langle \text{letra} \rangle \mid \langle \text{ID} \rangle \langle \text{digito} \rangle$
- ▶  $\langle \text{CTE} \rangle \rightarrow \langle \text{digito} \rangle \mid \langle \text{CTE} \rangle \langle \text{digito} \rangle$
- ▶  $\langle \text{letra} \rangle \rightarrow a \mid b \mid c \mid \dots \mid z$
- ▶  $\langle \text{digito} \rangle \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$

¿Esta gramática es regular?

# Identificadores

- **Identificador:** cadena que comienza con una letra y continúa con letras y/o dígitos (las letras pueden ser mayúsculas o minúsculas)



Gramática:

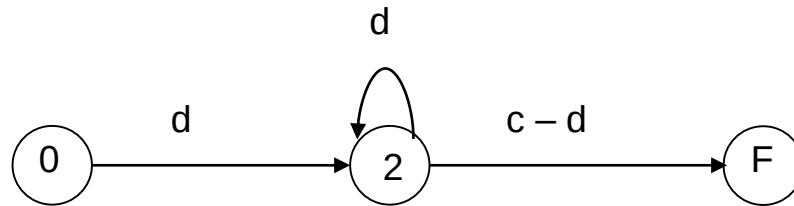
- 0: <Inicio>
  - 1: <ID>
  - F: <Fin>
- $\langle \text{ID} \rangle \rightarrow \langle \text{Inicio} \rangle l \mid \langle \text{ID} \rangle l \mid \langle \text{ID} \rangle d$   
 $\langle \text{Fin} \rangle \rightarrow \langle \text{ID} \rangle \text{ otro}$   
(otro es cualquier carácter distinto de l o d)

Expresión regular:

ID:  $l (l \mid d)^*$

# Constantes

- ▶ **Constante entera:** secuencia de dígitos



Gramática:

- ▶ 0: <Inicio>
- ▶ 2: <CTE>
- ▶ F: <Fin>

$\langle \text{CTE} \rangle \rightarrow \langle \text{Inicio} \rangle d \mid \langle \text{CTE} \rangle d$

$\langle \text{Fin} \rangle \rightarrow \langle \text{CTE} \rangle \text{ otro}$

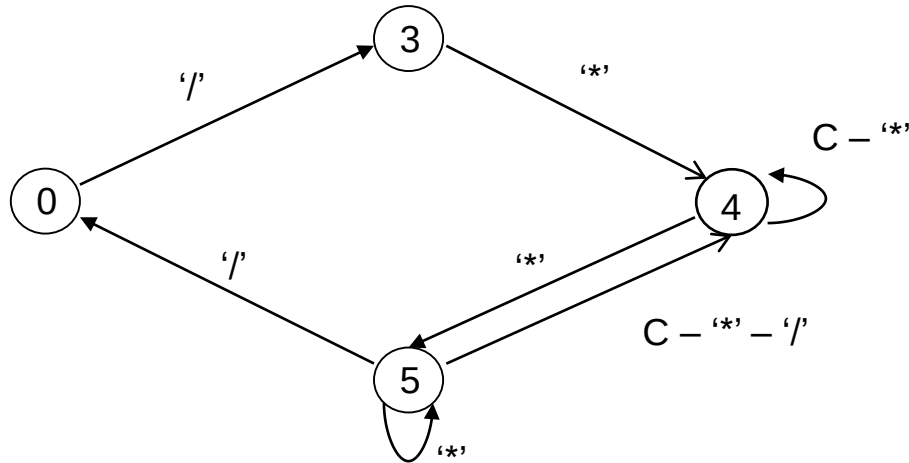
(otro es cualquier carácter distinto de d)

Expresión regular:

CTE:  $d^+$

# Comentarios

## ► Comentarios tipo C: /\* ... \*/



- 0: <Inicio>
- 3: <PCom>
- 4: <Com>
- 5: <PFCom>

<PCom> → <Inicio> '/'

<Com> → <PCom> '\*' | <Com> otro1 |

<PFCom> otro2

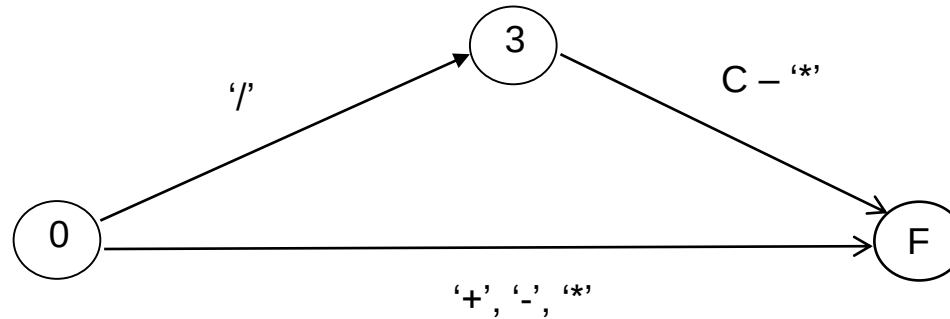
(otro1 es cualquier carácter distinto de '\*' y otro2 cualquier carácter distinto de '\*' y '/')

<PFCom> → <Com> '\*' | <PFCom> '\*'

<Inicio> → <PFCom> '/'

# Operadores

- ▶ Operadores aritméticos: + - \* /



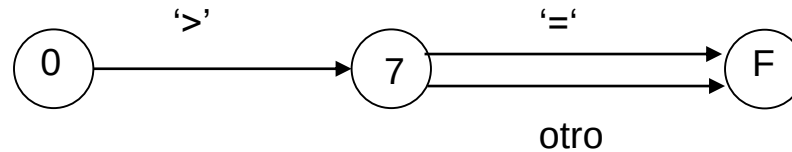
- ▶ 0: <Inicio>
  - ▶ 3: <PCom>
  - ▶ F: <Fin>
- $\langle \text{PCom} \rangle \rightarrow \langle \text{Inicio} \rangle \text{'/'}$
- $\langle \text{Fin} \rangle \rightarrow \langle \text{Inicio} \rangle \text{'+'} \mid \langle \text{Inicio} \rangle \text{'-'} \mid \langle \text{Inicio} \rangle \text{'*'} \mid \langle \text{PCom} \rangle \text{ otro1}$

(otro1 es cualquier carácter distinto de `'*'`)

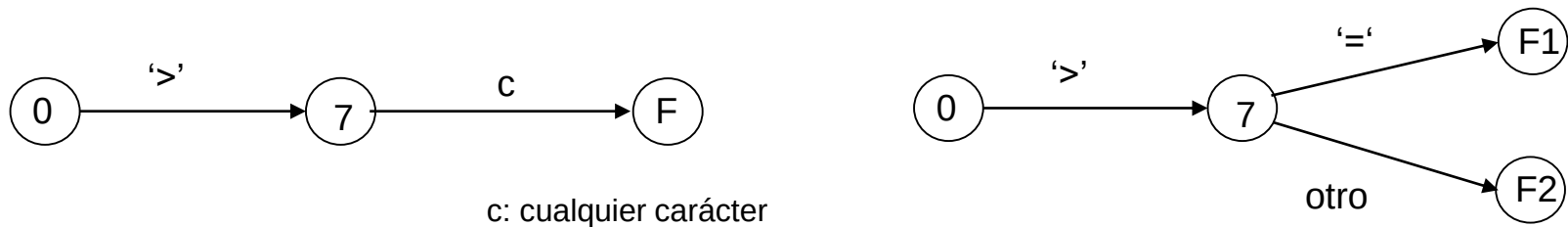
▶ )

# Comparadores

- ▶ **Comparadores:** > >= < <= == <>



- ▶ En Autómatas Finitos la representación debería ser diferente:

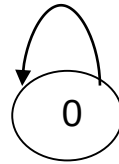


# Blancos, tabulaciones, saltos de línea

---

- ▶ Eliminar blancos, tabulaciones, saltos de línea

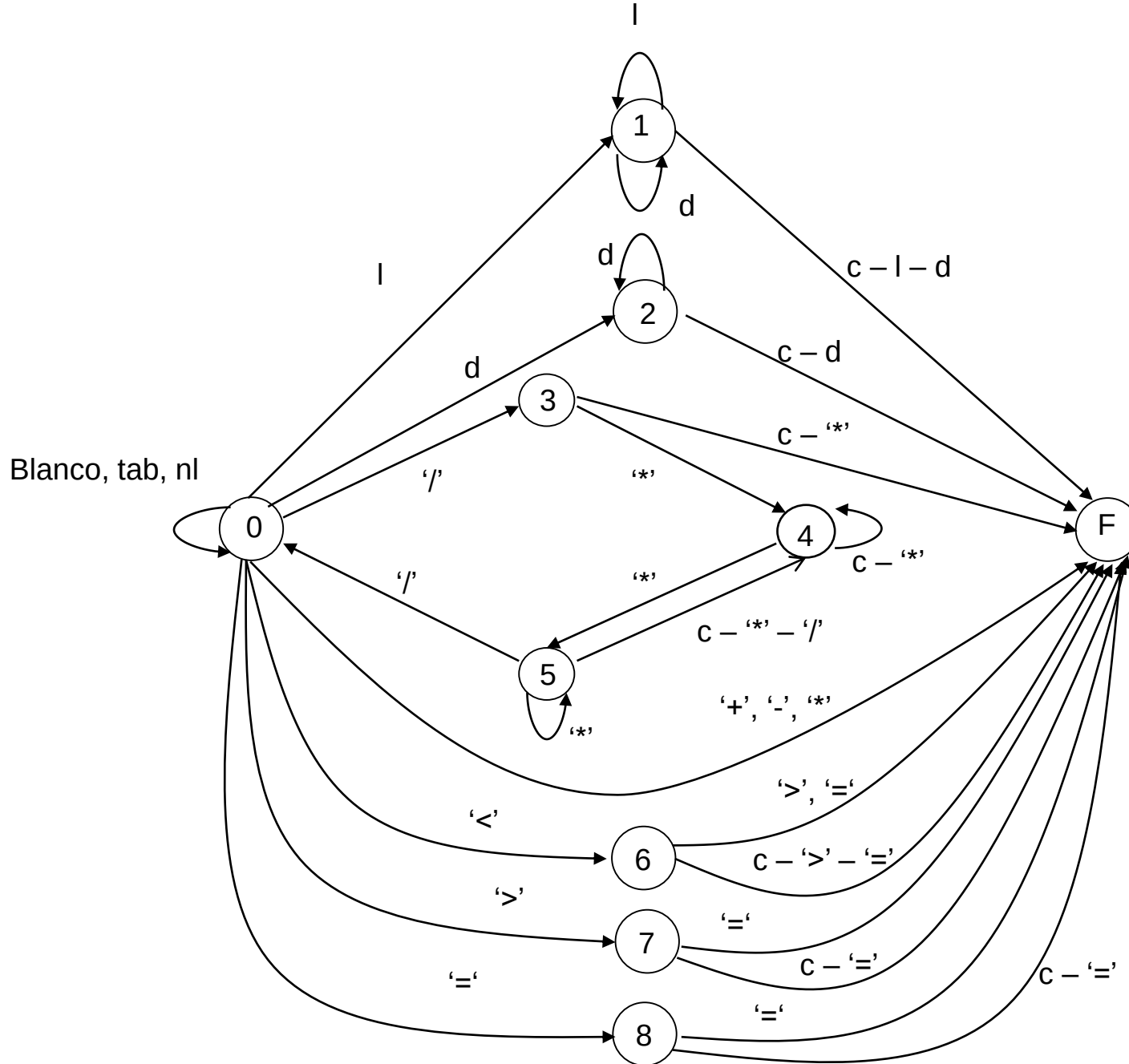
Blanco, tab, nl



- ▶ No se entregan al Analizador Sintáctico  
¿Nunca?

---

# Autómata para el Analizador Léxico



# Matriz de Transición de Estados

|   |   | d | / | * | + | - | = | < | > | otro | BL<br>tab nl | \$ |
|---|---|---|---|---|---|---|---|---|---|------|--------------|----|
| 0 |   | 2 | 3 | F | F | F | 8 | 6 | 7 | F    | 0            | F  |
| 1 |   |   | F | F | F | F | F | F | F | F    | F            | F  |
| 2 | F | 2 | F | F | F | F | F | F | F | F    | F            | F  |
| 3 | F | F | F | 4 | F | F | F | F | F | F    | F            | F  |
| 4 | 4 | 4 | 4 | 5 | 4 | 4 | 4 | 4 | 4 | 4    | 4            | F  |
| 5 | 4 | 4 | 0 | 5 | 4 | 4 | 4 | 4 | 4 | 4    | 4            | F  |
| 6 | F | F | F | F | F | F | F | F | F | F    | F            | F  |
| 7 | F | F | F | F | F | F | F | F | F | F    | F            | F  |
| 8 | F | F | F | F | F | F | F | F | F | F    | F            | F  |

---

¿Qué significa llegar al estado final?

# Analizador Léxico: Tokens

---

- ▶ Identificadores (cadena que comienza con una letra y continúa con letras y/o dígitos)
- ▶ Constantes enteras (secuencia de dígitos)
- ▶ Comentarios tipo C `/* ... */`
- ▶ Operadores aritméticos `+` `-` `*` `/`
- ▶ Comparadores `>` `>=` `<` `<=` `==` `<>`
- ▶ Palabras reservadas (secuencia de letras mayúsculas)

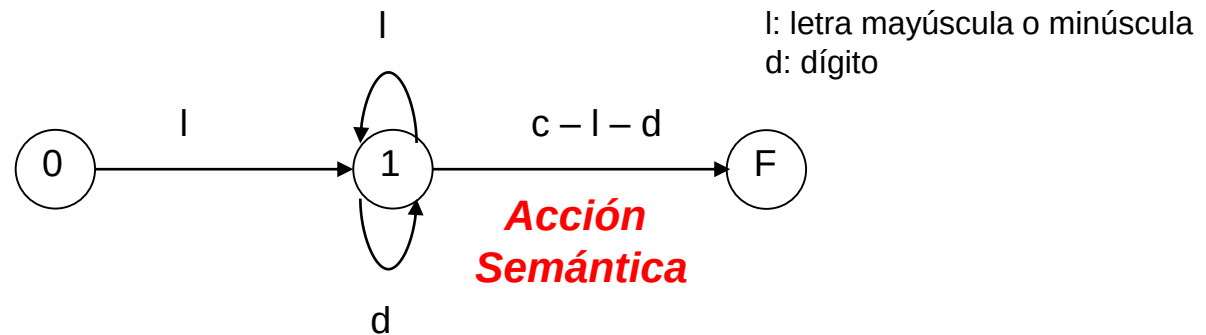
---

¿Y las palabras reservadas?



# Palabras Reservadas

- **Palabra reservada:** Secuencia de letras mayúsculas



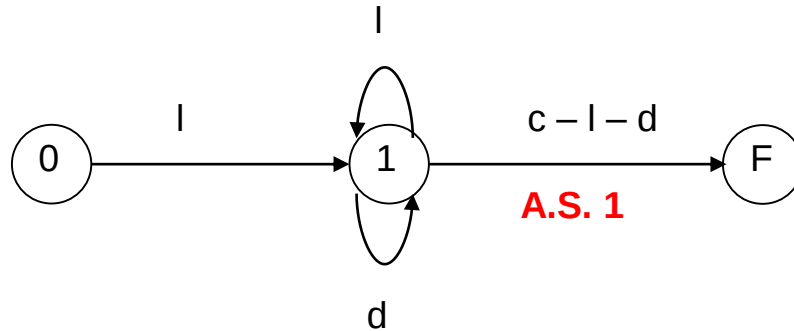
- ¿Identificador o palabra reservada?

# Acciones Semánticas

---

- ▶ Fragmentos de código en el lenguaje del compilador.
- ▶ Se asocian a las transiciones o a las expresiones regulares que definen tokens.
- ▶ Impiden el crecimiento del número de estados del autómata.

# Identificadores y Palabras Reservadas



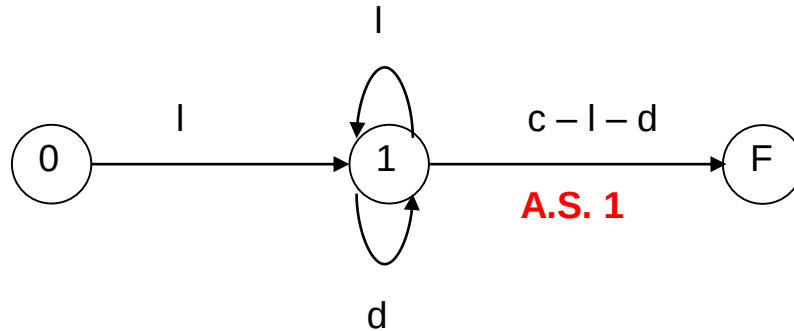
Almacenamiento de las palabras reservadas:

- ▶ Tabla de palabras reservadas
- ▶ Tabla de Símbolos

## ▶ Acción Semántica l:

- ▶ Devolver a la entrada el último carácter leído
- ▶ Buscar en la TPR
  - Si está, devolver la Palabra Reservada
  - Si no está,
    - Buscar en la TS
      - ▶ Si está, devolver ID + Punt TS
      - ▶ Si no está,
        - ▶ Alta en la TS
        - ▶ Devolver ID + Punt TS

# Identificadores y Palabras Reservadas



Almacenamiento de las palabras reservadas:

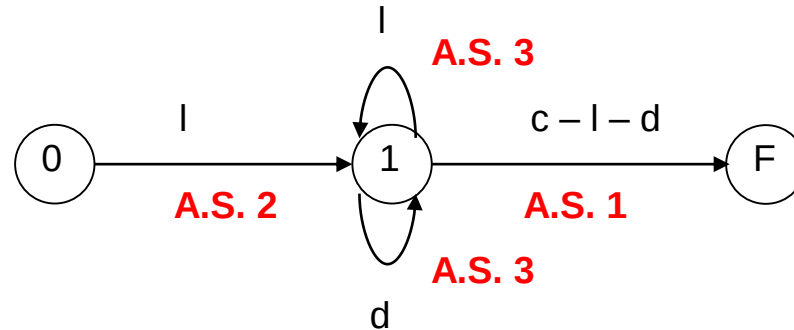
► Tabla de Símbolos

## ► Acción Semántica I:

- Devolver a la entrada el último carácter leído
- Buscar en la TS
  - Si está,
    - Si es PR, devolver la Palabra Reservada
    - Si no, Devolver ID + Punt TS
  - Si no está,
    - Alta en la TS
    - Devolver ID + Punt TS

# Acciones Semánticas

---



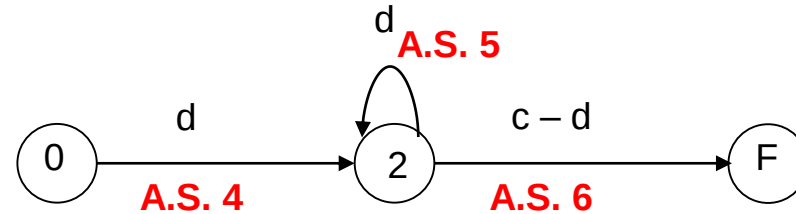
## ► Acción Semántica 2:

- Inicializar string (se reserva la máxima longitud permitida para identificadores)
- Agregar letra al string

## ► Acción Semántica 3:

- Agregar letra o dígito al string

# Constantes - Acciones Semánticas



- ▶ Acción Semántica 4:
  - ▶ Inicializar string para la constante
  - ▶ Agregar dígito al string
- ▶ Acción Semántica 5:
  - ▶ Agregar dígito al string
- ▶ Acción Semántica 6:
  - ▶ Devolver a la entrada el último carácter leído
  - ▶ Verificar rango de la constante
  - ▶ Alta en la TS
  - ▶ Devolver CTE + Punt TS

# Constantes

---

- ▶ **Formas de reconocerlas**
  - ▶ Un solo tipo de token (CTE)
  - ▶ Diferentes tipos de tokens (CTEINT, CTEFL)
- ▶ **Almacenamiento**
  - ▶ Tabla de Símbolos
  - ▶ Tabla de Constantes y Literales

# Constantes

---

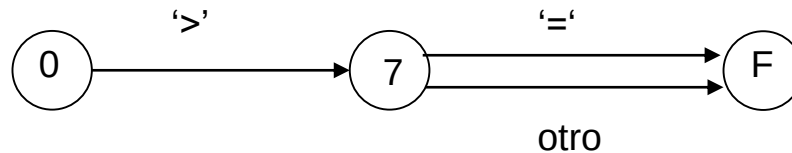
- ▶ Si mis constantes enteras pueden tomar valores entre: - -- -32768 y 32767:

¿Cómo reconozco constantes  
negativas?

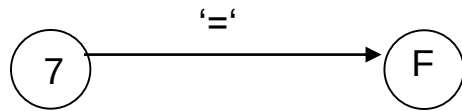


# Comparadores

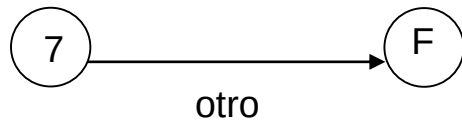
---



- ¿Qué diferencia hay entre una y otra transición?

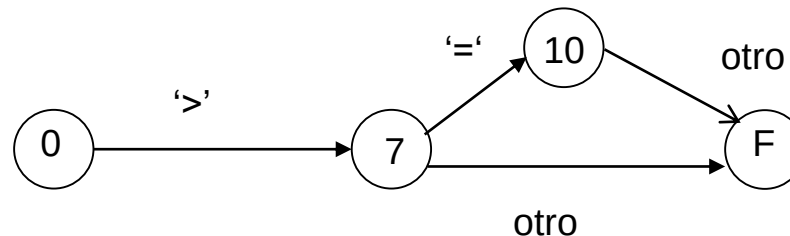


El carácter '=' se consume.



El carácter 'otro' no se consume.

- ¿Cómo se distingue una situación de la otra?
  - Solución 1:



- Solución 2: **ACCIONES SEMÁNTICAS**
- Solución para expresiones regulares: **Definir la expresión correspondiente a la cadena más larga primero.**

# Acciones semánticas

---

- ▶ ¿Cómo se asocian las acciones semánticas con las transiciones?

***Matriz de Transición de Estados***

(Contiene la “letra chica” del Análisis Léxico)

# Matriz de Transición de Estados

|   |          | d        | /        | *        | +        | -        | =        | <        | >        | otro     | BL<br>tab nl | \$       |
|---|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|--------------|----------|
| 0 | 1<br>AS2 | 2<br>AS4 | 3        | F        | F        | F        | 8        | 6        | 7        | F        | 0            | F        |
| 1 | 1<br>AS3 | 1<br>AS3 | F<br>AS1 | F<br>AS1 | F<br>AS1 | F<br>AS1 | F<br>AS1 | F<br>AS1 | F<br>AS1 | F<br>AS1 | F<br>AS1     | F<br>AS1 |
| 2 | F        | 2        | F        | F        | F        | F        | F        | F        | F        | F        | F            | F        |
| 3 | F        | F        | F        | 4        | F        | F        | F        | F        | F        | F        | F            | F        |
| 4 | 4        | 4        | 4        | 5        | 4        | 4        | 4        | 4        | 4        | 4        | 4            | F        |
| 5 | 4        | 4        | 0        | 5        | 4        | 4        | 4        | 4        | 4        | 4        | 4            | F        |
| 6 | F        | F        | F        | F        | F        | F        | F        | F        | F        | F        | F            | F        |
| 7 | F        | F        | F        | F        | F        | F        | F        | F        | F        | F        | F            | F        |
| 8 | F        | F        | F        | F        | F        | F        | F        | F        | F        | F        | F            | F        |

# Programación

---

- ▶ Para cada carácter leído, mapear con las columnas de la matriz
  - ▶ l → columna 0
  - ▶ d → columna 1
  - ▶ '/' → columna 2
  - ▶ ...
- ▶ Si el carácter leído no corresponde a ninguna columna, debe informarse “Carácter Inválido”

# Programación

---

- ▶ Definir matriz de estados

- ▶ nuevo\_estado [9][12] = {1,2,3,-1,-1,8,6,...}

# Programación

---

- ▶ Definir matriz de acciones semánticas

- ▶ `accion_sem [9][12] = {AS2, AS4, ... }`

Implementación:

Matriz de punteros a función o equivalente

# Programación

---

- ▶ Código para el Análisis Léxico:

- ▶ Datos:

- ▶ estado: estado actual
    - ▶ entrada: símbolo leído

- ▶ Acciones:

- ▶ accion\_sem[estado][entrada]  
ejecuta la acción semántica
    - ▶ estado = nuevo\_estado[estado][entrada]  
actualiza el estado actual

# Errores Léxicos

---

- ▶ En el mapeo de los símbolos de entrada, se pueden detectar caracteres inválidos.
- ▶ Si en un estado del autómata, el carácter de entrada no coincide con ninguno de los arcos de salida, se trata de un error.
- ▶ Constante fuera de rango.
- ▶ etc.

# Errores Léxicos

## Acciones frente a un error

---

- ▶ **Modo pánico:** descartar caracteres hasta que aparezca un token conocido

- ▶ Ej. Constante de punto flotante mal construida.

Consecuencias:

- ☐ Se informa como Error
  - ☐ El token se descarta

- ▶ **Inserción / Borrado / Reemplazo**

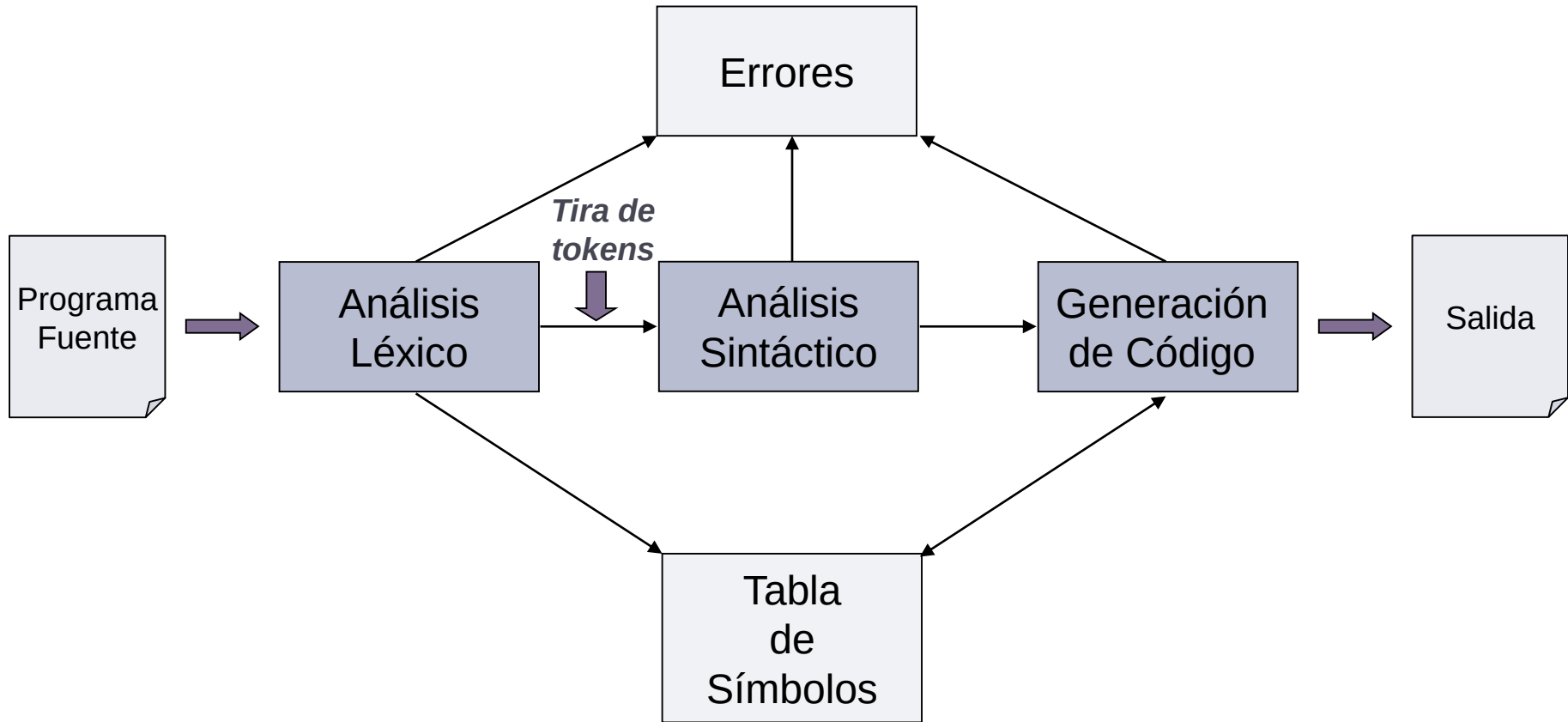
- ▶ Borrar un carácter de la entrada. Ej.: “:==”
  - ▶ Insertar un carácter omitido. Ej.: “:”
  - ▶ Sustituir un carácter por otro. Ej.: “:+”
  - ▶ Transponer dos caracteres adyacentes. Ej.: “=:”

Consecuencias:

- ☐ Se informa como Warning
  - ☐ El token (modificado) se entrega como token válido

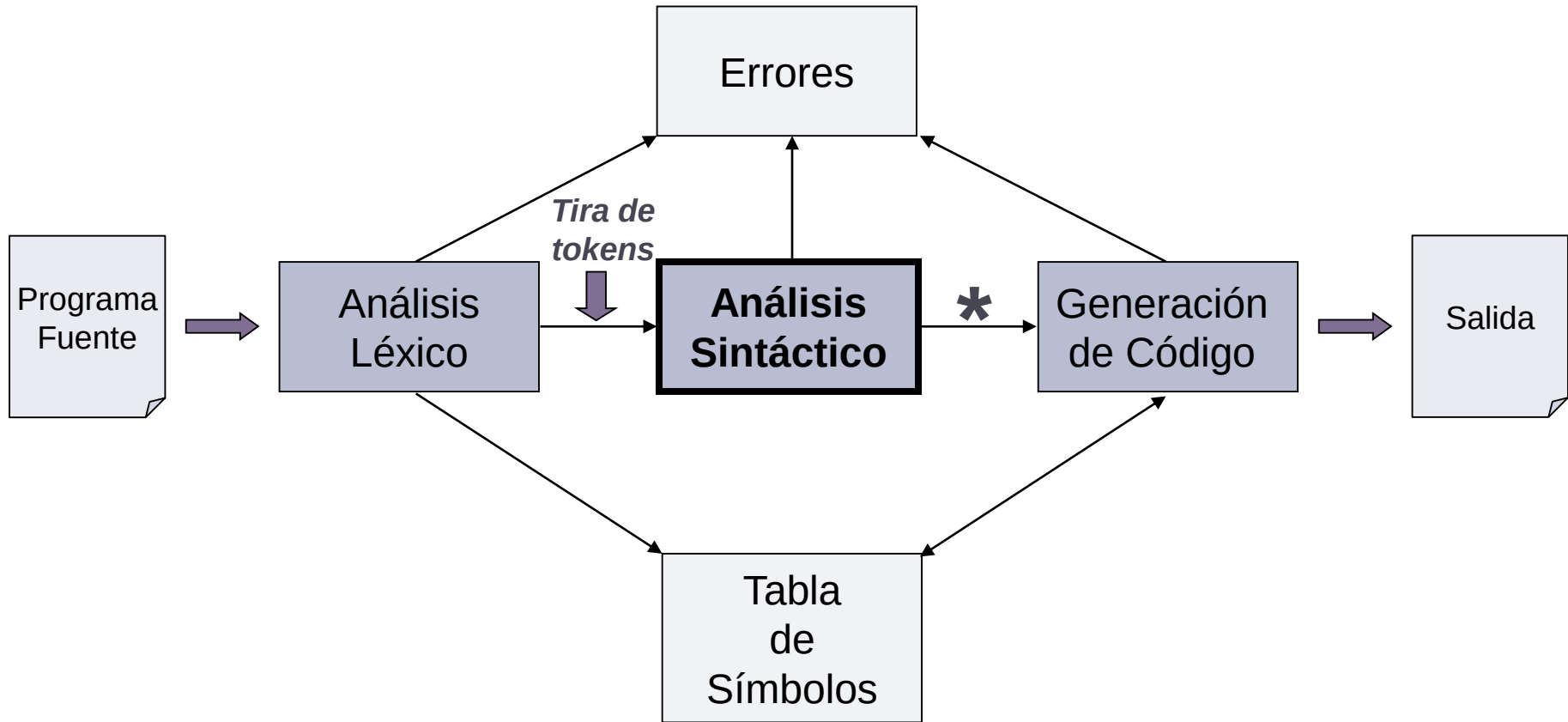
# Fases de la Compilación

---



# Fases de la Compilación

---



---

¿Preguntas?

